

Software Engineer, Intelligence

Team: Intelligence · **Experience:** 0–2 years · **Location:** [to be filled] · **Type:** Full-time

About SmartJoules

SmartJoules exists to eliminate energy waste in India's buildings. We instrument hospitals, offices and hotels with our own hardware, stream what we learn to the cloud, and close the loop by sending optimised control decisions back to the equipment. Every percentage point of efficiency we find is energy that never has to be generated.

The Intelligence team builds the software and the models behind that loop — from control software running in plant rooms, to the pipelines and models that make sense of what buildings tell us, to the AI systems that put those insights in front of the people who operate them.

About the role

As a Software Engineer on the Intelligence team, you will work across the systems that turn raw sensor readings into decisions: data pipelines processing telemetry from thousands of pieces of equipment, machine learning models that predict how that equipment behaves, and an AI agent that answers operators' questions from live building data.

This is a deliberately broad role. In your first year you will touch data engineering, applied machine learning and backend work, and you will find out which of them you enjoy most. You will work alongside senior engineers who review your code and explain their reasoning, and with product managers and customers who will tell you directly whether what you built is useful.

We do not expect you to know anything about buildings or HVAC when you start. Most of us did not.

Our tech stack

- **Languages** — Python (primary), JavaScript, TypeScript
- **Machine learning** — scikit-learn, XGBoost, LightGBM, Keras, pandas, numpy, AWS SageMaker
- **Reinforcement learning and control** — Model Free RL, Deep RL, ML based Optimiser.
- **AI systems** — AWS Bedrock, tool-calling agents, retrieval-augmented generation
- **Data platform** — Apache Airflow, InfluxDB, PostgreSQL, Redis, S3
- **Knowledge graph** — Amazon Neptune, Brick schema
- **Observability** — Grafana, Prometheus, Loki, Sentry, OpenTelemetry

- **Backend and front end** — FastAPI, Node.js, Angular
- **Edge** — ARM Linux, MQTT
- **Cloud and infrastructure** — AWS, Docker, GitHub Actions

What you will do

- Explore and clean real sensor data — the kind with gaps, drift, stuck values and units that disagree — and work out what it is actually telling you.
- Build and maintain data pipelines that turn raw equipment telemetry into the metrics our models and our product depend on.
- Train, evaluate and retrain machine learning models on plant sensor data — power draw, water flow, temperatures, and the efficiency baselines used to verify savings
- Write tools for our AI agent: functions that fetch data from our knowledge graph and time-series stores and return it in a form a language model can reason over
- Trace data problems across the stack, from a sensor on a machine through message transport, storage and APIs to the dashboard someone is looking at
- Write tests, particularly for the cases that have already caught us out once
- Demonstrate your work — to the team regularly, and to customers as you grow into it
- Visit sites. Seeing the equipment your code affects will change how you build for it

You may be a good fit if you have

- Comfort with Python, and the ability to read and modify code you did not write. Familiarity with pandas and numpy
- **Statistics fundamentals** — distributions, mean against median, variance, correlation, outliers, and the difference between a correlation and a cause. Enough to know when a number is telling you something and when it is noise
- **Exploratory data analysis as a habit.** Given an unfamiliar dataset, you plot it, look at its shape, find what is missing, and form questions before you model anything
- **Machine learning fundamentals** from projects you have built end to end, through coursework, an internship or on your own — regression and classification, train and test splits, cross-validation, overfitting, feature engineering, and the ability to read an evaluation metric and explain what it means
- Familiarity with scikit-learn. Exposure to XGBoost, LightGBM, or a deep learning framework such as PyTorch, TensorFlow or Keras is a bonus
- Working knowledge of SQL and basic data modelling
- Comfort with Git, the command line and a Linux environment
- A habit of checking your own work. If you report a result, we will ask how you verified it
- Willingness to explain what you built to people who do not write code
- Curiosity about a domain you do not yet know. The physics of a chiller plant is learnable; the interest in learning it is what we look for

Nice to have

- Any project built with large language models — retrieval-augmented generation, an agent with tool use, a chatbot over your own documents. Built with LangChain, LlamaIndex, AWS Bedrock, or direct API calls; coursework and side projects count
- Neural networks and deep learning, in any framework, applied to any problem
- An understanding of what reinforcement learning is and where it fits — agents, states, actions and rewards. We run RL in production, and you will work alongside it
- Open source contributions of any size — a merged pull request, a library you maintain, or a package you published
- Projects involving time-series data or IoT
- Exposure to AI coding assistants such as Claude Code, Codex, Cursor or Copilot. We use them daily, and we will teach you how we review what they produce
- Docker, and any cloud platform
- Any hardware or control-systems project, academic or personal
- Experience working directly with users or customers, in any role

Logistics

- **Minimum education** — Bachelor's degree, or an equivalent combination of education, training and experience
- **Experience** — 0–2 years. Recent graduates with strong internships or substantial personal projects are encouraged to apply
- **Location** — [to be filled]
- **Compensation** — [to be filled]

We encourage you to apply even if you do not believe you meet every qualification listed. Strong candidates rarely match every line of a job description, and we would rather read your application than have you rule yourself out.

How we work

Our software has physical consequences. When we get something wrong, a hospital ward gets uncomfortable or a chiller runs when it should not. That shapes how we work: we measure before we claim, we write down why a decision was made, and we treat "I am not sure" as a useful thing to say.

We also work openly with the rest of the company. Engineers here sit with product managers, talk to the people who run these buildings, and present their own work rather than having it presented for them. Understanding why something matters is part of the engineering, not something that happens afterwards.

